

Network Routing Optimization

Shortest Path Finding

Team Members:

Bryce Lander

Steven Crocker

Ricky Newbold

CSC 2400: Design of Algorithms

Overview:

Problem: Finding the shortest possible route to implement network routing optimization.

Methods: Dijkstra's algorithm, A* algorithm, Bellman-Ford algorithm, Ant Colony Optimization algorithm.

Objective/Research:

1. How does the runtime of Algorithm A, B, and C grow as input size n increases?

As the number of nodes in the graph increases (usually in bigger cities), the runtime of all algorithms increases. Dijkstra's algorithm shows steady growth, since by nature it explores the graph in a structured way from the starting point. A* grows more slowly as it uses a heuristic approach to guide the search toward the destination and reduce unnecessary exploration. Bellman-Ford's runtime increases more rapidly because it has to repeat processes on all edges in the graph. ACO (Ant Colony Optimization) shows the highest increase in runtime, as it relies on multiple iterations of exploration rather than a direct calculation. Overall, we can conclude that the runtime growth is influenced by node size; larger node sizes lead to higher runtimes, with the rate of growth depending on how efficiently each algorithm handles it.

2. Which algorithm scales best as n increases?

A* scales the best as the number of nodes (n) increases. As the graph becomes larger, its runtime and memory usage grow at a relatively controlled rate, allowing it to remain efficient on more complex networks. This is due to its use of heuristics, which help guide the search towards the destination and reduce unnecessary exploration. Dijkstra's algorithm behaves similarly in that it also produces reliable results, but it does not use this additional guidance, which leads to more work as n increases. Overall, A* maintains better efficiency as the graph size grows.

3. How does runtime scale versus memory usage?

As the number of nodes increases, both runtime and memory usage increase, but the amount of memory used depends on how efficiently each algorithm processes the graph. More efficient algorithms, such as A* and Dijkstra, can keep memory usage relatively low because they store only the necessary information needed to track distances and paths during the search. In contrast, algorithms that perform more repeated computations or maintain additional data structures require more memory. For example, ACO shows significantly higher memory usage because it must store multiple paths and pheromone values through its iterations. This demonstrates that algorithms that minimize unnecessary exploration and data storage tend to scale better in runtime and memory as the graph size increases.

Description of Experiments:

Graph Attribute Effect Benchmark:

Description:

We used Python to implement the code and created a html file to have a visual interface for the comparisons. We ran the comparison for 20 cites and compared 5 attributes (node count, edge count, average degree, circuitry average, and intersection count) against each algorithm's runtime. ACO was only able to find a route 6 out of the 20 runs. We then repeated the comparison looking at memory usage instead of runtime.

Computer Characteristics:

CPU: Intel Core i7 8700

Memory: 64GB DDR4

GPU: GTX 1660 TI

Storage: N/A

Performance Metric:

Algorithm	Average Execution Time in Seconds	Basic Operations	How close it is to the optimal solution
Dijkstra's	0.231	Comparison	Dijkstra is the reference algorithm for the optimal route. Since it guarantees the shortest path using the graph's travel-time weights, its result is the baseline.
A*	0.088	Comparison	A* was also able to find the optimal route in most cases. Since it uses a heuristic approach while still being optimal, its cost usually matches Dijkstra's result, sometimes faster. Which means A* had about a 0-13.6% optimal solution.

Bellman-Ford	19.95	Addition & Comparison	Bellman-Ford also produced routes that matched the optimal solution when ran successfully. Due to it also be a shortest- path algorithm its travel time mostly matched Dijkstra's baseline. Its optimal gap was in most cases, around 0%, but it required much more runtime and memory to get there.
ACO	256.03	Probability	ACO often produced routes that had a much longer travel time and were much more inefficient compared to other algorithms. Most often it was unable to find a route.

Scalability Benchmark:

Description:

Utilizing the OSMnx python package I used python to implement a large benchmark test implementing all our algorithms in 150 OSM MultiDiGraphs (110 successful measurements taken) to be able to determine the algorithm scalability as the graph size (input size) increased. This helped show which algorithms (Dijkstra's & A*) were most optimal for solving the shortest path being able to accurately and reliably find the shortest path while also meeting optimal standards for runtime and memory usage. This data showed how Dijkstra's Algorithm was best for the shortest path while still have satisfactory runtime and memory usage while A* Algorithm was the best algorithm to implement for finding the shortest path in network routing when you are looking to optimize for the fastest runtime and memory usage which is especially important in resource constrained environments.

Computer Characteristics:

CPU: Intel Core Ultra 9 185H

Memory: 32GB DDR5 7467MT/s

GPU: Intel Arc Graphics

Storage: 2TB M.2 NVME SSD

Performance Metric:

Algorithm	Average Execution Time in Seconds	Basic Operations	How close it is to the optimal solution
Dijkstra's	2.72	Comparison	Dijkstra is the reference algorithm for the optimal route. Since it guarantees the shortest path using the graph's travel-time weights, its result is the baseline.
A*	0.59	Comparison	A* was also able to find the optimal route in most cases. Since it uses a heuristic approach while still being optimal, its cost usually matches Dijkstra's result, sometimes faster. Which means A* had about a 0-13.6% optimal solution.

Bellman-Ford	197.49	Addition & Comparison	Bellman-Ford also produced routes that matched the optimal solution when ran successfully. Due to it also be a shortest- path algorithm its travel time mostly matched Dijkstra's baseline. Its optimal gap was in most cases, around 0%, but it required much more runtime and memory to get there.
ACO	35.63	Probability	ACO often produced routes that had a much longer travel time and were much more inefficient compared to other algorithms. Most often it was unable to find a route.

GPS Routing:

Description:

We used a Raspberry Pi Zero 2 W to create a resource constrained environment. We then implemented a GPS program to test our algorithms capability in a real world application such as utilizing network routing optimization to find the shortest path to then be able to successfully implement them toward GPS Routing in a resource constrained environment as many GPS Device will likely be low power/resource devices and would require such optimizations to be able to be made such to be able to run quickly and optimally with little to no issues despite the hardware constraints. This was able to determine that Dijkstra's and A* algorithms were the only applicable Algorithms for this application, specifically A* being the most optimal algorithm overall due to its fastest runtime and lowest memory usage (memory usage proved in previous experiments) while also having either equal to within 10% path cost gap to the shortest possible path.

Computer Characteristics:

CPU: Quad-Core 64-bit Arm Cortex-A53 CPU

Memory: 512MB LPDDR2 SDRAM + 1GB SWAP microSD memory

GPU: Broadcom VideoCore IV GPU

Storage: 32GB microSD

Performance Metrics:

Algorithm	Average Execution Time in Seconds	Basic Operations	How close it is to the optimal solution
Dijkstra's	1348.67ms	Comparison	Dijkstra is the reference algorithm for the optimal route. Since it guarantees the shortest path using the graph's travel-time weights, its result is the baseline.
A*	1066.33ms	Comparison	A* was also able to find the optimal route in most cases. Since it uses a heuristic approach while still being optimal, its cost usually matches Dijkstra's result, sometimes faster. Which means A* had about a 0-13.6% optimal solution.

Bellman-Ford	49950ms	Addition & Comparison	Bellman-Ford also produced routes that matched the optimal solution when ran successfully. Due to it also be a shortest- path algorithm its travel time mostly matched Dijkstra's baseline. Its optimal gap was in most cases, around 0%, but it required much more runtime and memory to get there.
ACO	NO PATH FOUND	Probability	ACO often produced routes that had a much longer travel time and were much more inefficient compared to other algorithms. Most often it was unable to find a route.

Description of Work:

Task	Name	Percent
Brainstorming	Bryce Lander	BL - 100%
Coding	Bryce Lander, Steven Crocker	SC - 30%, BL - 70%
Experiment Design	Bryce Lander	BL - 100%
Data Collecting	Steven Crocker, Bryce Lander	SC - 30%, BL - 70%
Visualization	Bryce Lander	BL - 100%
Report Writing	Bryce L, Steven C, Ricky N	BL - 33%, SC - 33%, RN - 33%
Presentation	Bryce L, Steven C, Ricky N	BL - 50%, SC - 25%, RN - 25%
Youtube Video	Bryce Lander	BL - 100%

Difficulties:

Most of the difficulty in this project came from time and scheduling restraints. One big roadblock found was making sure all necessary packages were installed and getting the OSMnx environment set up. When implementing the GPS Routing on my self-made GPS device, having the correct Python packages installed was a big initial issue since I was not able to install OSMnx traditionally due to hardware compatibility issues and had to do so through the “pip” install. This led to many necessary packages not installed and having to install them as discovered to alleviate errors. My largest roadblock I found while implementing the GPS specifically was optimizing graph loading specifically. The algorithms I had optimized for the device prior to testing and work seamlessly. However the graphs could not be loaded traditionally into ram due to the low 512MBs on a pi zero 2 w. This led to me learning new ways to load the graph in subgraph parts and stripping graphs of all unnecessary data besides what was needed for the algorithms. That way guaranteed graph loading would not be as intensive on memory usage. It then allowed me to test the algorithms on the smaller subgraph parts using only what they needed to which helped increase the speed and even efficiency of the program as well as the algorithms greatly. When implementing the scalability benchmark test, I initially wanted to make the test scale to the size of the US graph to showing scalability to a super large scale. Unfortunately, I was limited by my hardware and was not able to get the graph loaded so I set my node cap to 10 million. Later after this graph was able to load it was optimized down to have a total node count ~1.2 million nodes within a .pickle graph to be best optimized for the algorithms to be tested on it. Otherwise, the testing went smoothly.

Sources:

Benchmarking:

<https://claude.ai/share/fa64f6c2-3444-4b2e-8e2c-d9515c499b55>

Context: Claude helped with creating the scalability benchmark.

<https://claude.ai/share/1df6059f-c172-45b0-9297-a5b10cf86f67>

Context: Claude helped with creating the GPS program to benchmark algorithms on low power/resource devices to achieve GPS Routing through their shortest pathing.

<https://pip-assets.raspberrypi.com/categories/584-raspberry-pi-zero-2-w/documents/RP-008359-DS-1-raspberry-pi-zero-2-w-product-brief.pdf?disposition=inline>

Context: Documentation for Pi Zero 2 W, specifically helpful for pinout reference.

<https://chatgpt.com/share/69eeba44-2a84-83e8-9dee-42b6d772a6ac>

Context: ChatGpt helped with automation for running 20 cities at one time, as well as OSMnx mapping of algorithms.

<https://www.matts-electronics.com/wp-content/uploads/2018/06/MP1584EN.pdf>

Context: Documentation for MP1584EN Buck Converter I used to find ideal input voltage and pinout reference.

<https://matplotlib.org/stable/Matplotlib.pdf>

Context: Documentation for Matplotlib which helped with graphing and plotting our data.

OSMnx Installation/Documentation:

<https://osmnx.readthedocs.io/en/stable/installation.html>

Context: Gives all info on how to install OSMnx and great documentation whats in the python package.

<https://docs.conda.io/en/latest/>

Context: Install for miniConda(to install OSMnx) and documentation to help with install.

<https://networkx.org/documentation/stable/reference/index.html>

Context: Has great documentation for how networkx functions work to be able to better utilize them.

Results:

Overview:

We took MultiDiGraphs from the OSM dataset to test Dijkstra's, A*, Bellman-Ford, and Ant-Colony Optimization to implement Network Routing Optimization through the problem of finding the shortest possible path.

We did this through three separate experiments. We tested the effects of graph attributes on algorithm efficiency. This allowed us to determine that node count aka the graph size was the largest determining factor for algorithm efficiency and any other affecting attributes directly correlated with graph size.

We then ran a large scalability benchmark to test all the algorithms efficiency as graph size (input size) increases. This allowed us to determine that Dijkstra's and A* were by far the most optimal algorithms for finding the shortest possible path to best optimize network routing. This also allowed us to determine bellman-ford was not a good use case for this problem and is better suited for graphs with negative edges which with our datasets OSM road network MultiDiGraphs do not have negative edges although while it had non-optimal runtime and exponential memory usage although it was able to match guarantee a shortest path like Dijkstra's. ACO was all around not a good algorithm for this problem struggling greatly to find a path in these graphs due to the nature of OSM road networks and their structure leading to ants to struggle to find paths getting stuck looping usually.

From these benchmarks I was able to determine that A* and Dijkstra's were the only Algorithms able to be truly implemented for a GPS Routing System in a resource limited device such that a GPS device would likely be. I used a Raspberry Pi Zero 2 W to implement this resource limited device for the GPS Program to run on and experiment with how the algorithm's efficiency is affected in low resource environments while also optimizing graph loading and algorithms to be best optimized for such an environment. Due to the limitations I only allowed Bellman-Ford and ACO algorithms to run on smaller subgraphs due to their high memory usage, long runtime, and for ACO pathfinding struggles. This allowed me to efficiently display directly how to implement and optimize algorithms and the graphs they work with and their ability to be used in real work applications such as GPS Routing for low power/resource devices that require precise and intensive optimization to be able to run and operate as intended for the user.

.csv Table Results:

 **benchmark_results**

 **20 Cities: Memory,Nodes,Runtime**

 **20 Cities: Gps Pathing**

Graphs:

We have lots of graph data so I have included all our github repo's file paths to all graphs taken from our experiments.

Scalability Benchmark Graphs:

~\Network-Routing-Optimization\Graphs_Archive\scalability_path_benchmark\Scalability_Benchmark_Graphs

Graph Attribute Benchmark Graphs:

~\Network-Routing-Optimization\Graphs_Archive\Data for 20 cities\Graphs

GPS Tracking & Re-Routing Photos

~\Network-Routing-Optimization\Graphs_Archive\GPS Routing\GPS tracking and rerouting photos

GPS Routing

~\Network-Routing-Optimization\Graphs_Archive\GPS Routing\GPS Algo Screenshots

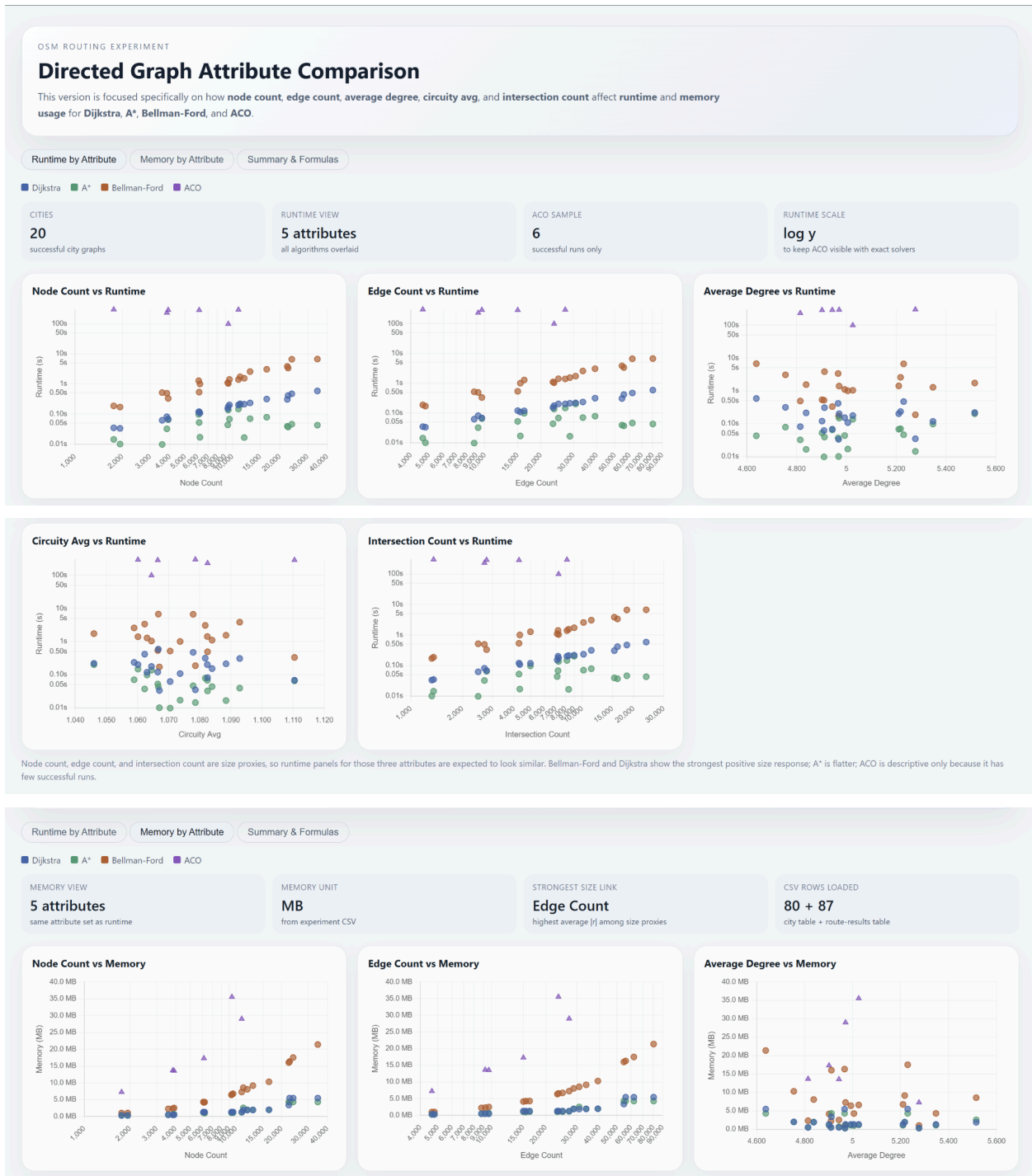
GPS Routing Videos

~\Network-Routing-Optimization\Graphs_Archive\GPS Routing\GPS Routing Videos

Youtube GPS Routing Video Link (this video link is also in previous file path just above as well)

<https://youtu.be/VjgA1pkiBkc>

graph_attribute_effects_dashboard.html:



Node Count vs Memory

Edge Count vs Memory

Average Degree vs Memory



Memory is much more tightly linked to graph size than to degree or circuitry. In this dataset, size-driven memory growth is visible across all algorithms, including A* and Bellman-Ford.

Runtime by Attribute

Memory by Attribute

Summary & Formulas

NODE RANGE

1,759-34,582

graph nodes

EDGE RANGE

4,641-80,194

graph edges

AVG DEGREE RANGE

4.638-5.515

narrow spread

CIRCUITY RANGE

1.046-1.110

narrow spread

INTERSECTION RANGE

1,321-23,614

graph intersections

ROWS USED

66

successful algorithm-city runs

Primary source table
city_network_experiment_results.csv provides the attribute and performance rows used in the comparison.

Supplemental source table
all_city_algorithm_results.csv is loaded as the paired route-results file you provided and used for source completeness reporting.

Average degree
 $k_{avg} = 2 * |E| / |V|$

Directed within-algorithm comparison formula
 $Pearson_r = CORREL(FILTER(outcome_range, algorithm=alg), FILTER(attribute_range, algorithm=alg))$

Recommended regression form for size effects
 $LN(outcome) \sim attribute + algorithm_dummies$

Important caveat
Node count, edge count, and intersection count are highly collinear in this dataset
 $r(nodes, edges) = 0.997$, $r(nodes, intersections) = 0.996$

Runtime Correlations (Pearson r)

Algorithm	n	Node Count	Edge Count	Average Degree	Circuitry Avg	Intersection Count
Dijkstra	20	0.985	0.987	-0.212	-0.144	0.989
A*	20	0.045	0.075	0.461	-0.486	0.090
Bellman-Ford	20	0.950	0.958	-0.152	-0.073	0.959
ACO	6	-0.395	-0.403	0.103	0.285	-0.432

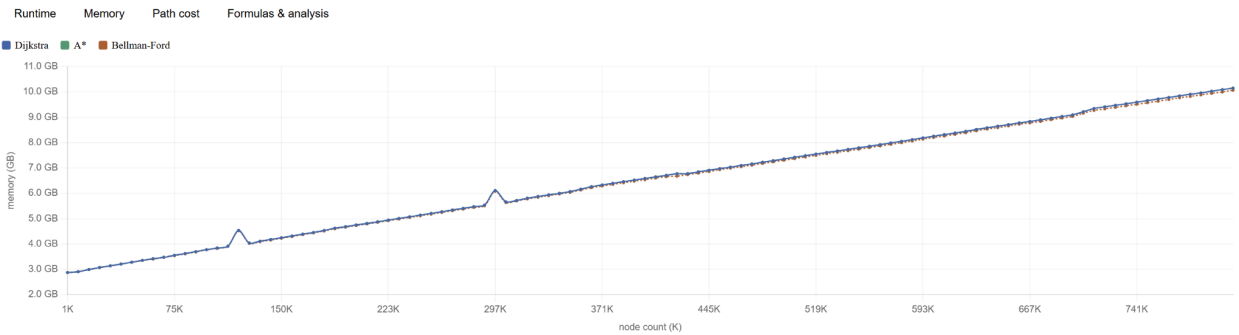
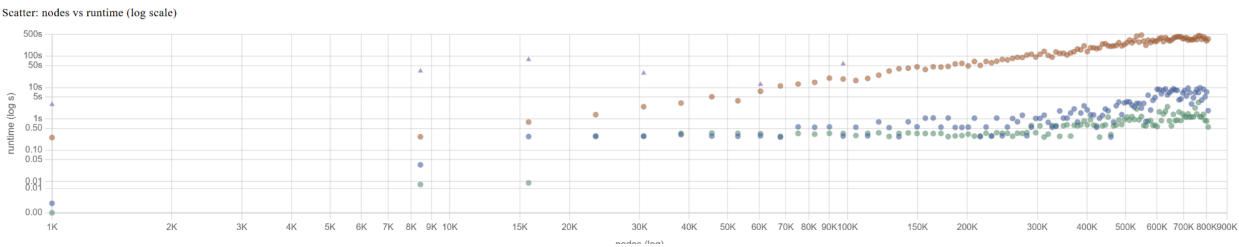
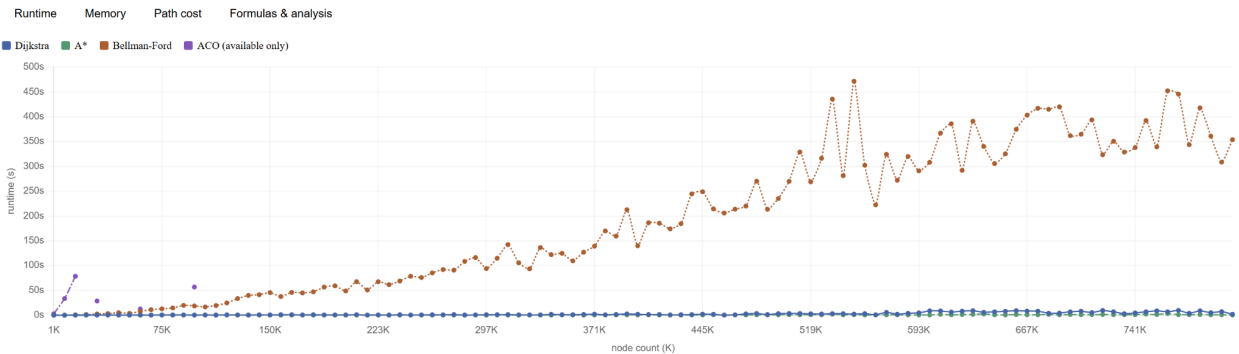
Memory Correlations (Pearson r)

Algorithm	n	Node Count	Edge Count	Average Degree	Circuitry Avg	Intersection Count
Dijkstra	20	0.939	0.951	-0.139	-0.141	0.947
A*	20	0.939	0.955	-0.093	-0.104	0.947
Bellman-Ford	20	0.992	0.998	-0.193	-0.103	0.996
ACO	6	0.939	0.943	-0.194	-0.596	0.953

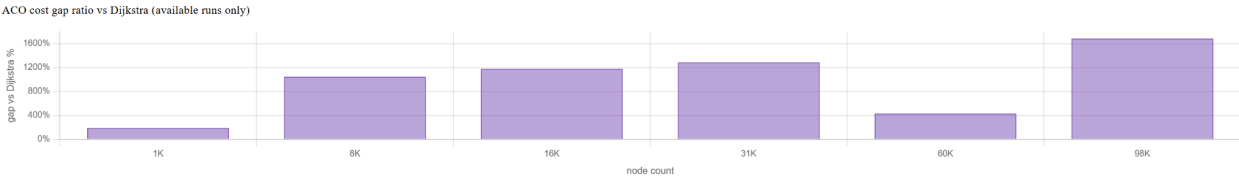
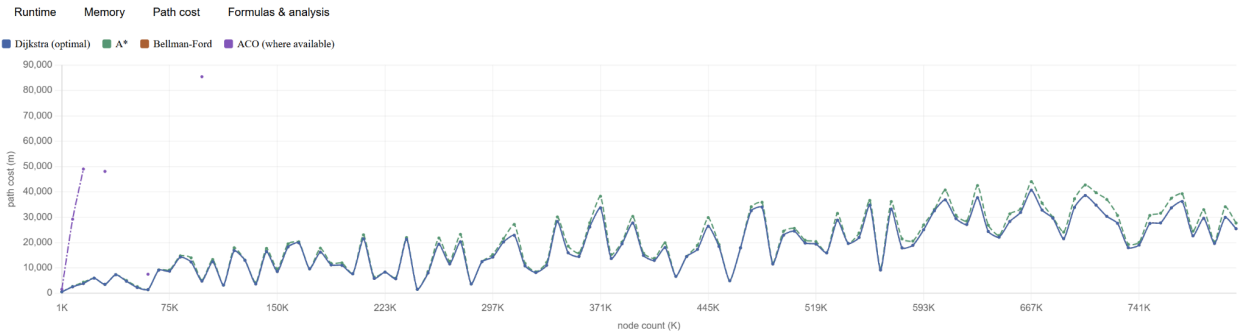
Use the runtime and memory tabs for the visual comparison, and use these tables when you need the exact within-algorithm direction and strength of each attribute relationship. ACO coefficients are much less stable because only 6 runs succeeded.

scalability_benchmark_dashboard.html:

Interactive dashboard comparing Dijkstra, A*, Bellman-Ford, and ACO algorithm performance on OSM road networks across 110 node-scale steps from 1,000 to 806,946 nodes.



ACO memory omitted (no data for scale >97K nodes). All three exact algorithms show nearly identical memory growth — memory is dominated by graph structure, not algorithm overhead.



Runtime	Memory	Path cost	Formulas & analysis		
GRAPH ATTRIBUTE RANGE		AVG NODE DEGREE		GRAPH CIRCUITRY	ACO AVAILABILITY
1K–807K		5.47–5.63		2.73–2.87	12 / 110
nodes (Kansas OSM)		stable across scale		Avg_Edges / Avg_Nodes	runs found path

Graph attribute formulas

Graph density (sparse-to-dense ratio)
 $\rho = 2 \cdot |E| / (|V| \cdot (|V|-1)) \rightarrow$ approaches 0 for large sparse road graphs

Avg node degree (connectivity)
 $\bar{k} = 2 \cdot |E| / |V|$ (undirected) ≈ 5.47 -5.63 across all scales

Graph circuitry index (cycle richness)
 $\gamma = |E| / (3 \cdot (|V|-2)) \rightarrow 0$ = tree, 1 = fully triangulated

Intersection density
 $I_density = Intersection_Count / |V| \approx 0.784$ (stable ratio)

Runtime complexity formulas

Dijkstra (with binary heap)
 $T(V,E) = O((V + E) \cdot \log V) \rightarrow$ empirical fit: $T \approx \alpha \cdot (V \cdot \log V) + \beta \cdot E \cdot \log V$

A* (with admissible heuristic h)
 $T(V,E) = O((V + E) \cdot \log V) \rightarrow$ same worst-case, but expands far fewer nodes.
Empirical advantage: $T_A^* \approx T_Dijkstra / k$ where $k \approx 2$ -30 $\times$ (heuristic quality)

Bellman-Ford
 $T(V,E) = O(V \cdot E) \rightarrow$ empirical fit: $T \approx \alpha \cdot V \cdot E$
At 500K nodes $\times$ 1.4M edges: $V \cdot E \approx 7 \times 10^{11}$ — explains 300-470s runtimes

ACO (Ant Colony Optimization)
 $T = O(n_ants \cdot n_iter \cdot |E|)$ — stochastic, parameter-sensitive
Fails on large graphs due to pheromone matrix dimensionality

Memory complexity formulas

All three exact algorithms
 $M(V,E) = O(V + E) \rightarrow$ empirical fit: $M \approx 2.87 + c \cdot V$ [GB]
Regression intercept (base process overhead) ≈ 2.87 GB
Slope $\approx 9.1 \times 10^{-6}$ GB/node (≈ 9.1 KB per node including adjacency lists)

A* additional memory
 $M_A^* = M_Dijkstra + O(\text{open set}) \approx$ identical in practice

ACO
 $M_ACO = O(|E|)$ for pheromone matrix $\rightarrow$ prohibitive at large scale

Path cost quality formulas

Optimality gap vs Dijkstra (baseline)
 $Gap\% = (Cost_alg - Cost_Dijkstra) / Cost_Dijkstra \times 100$

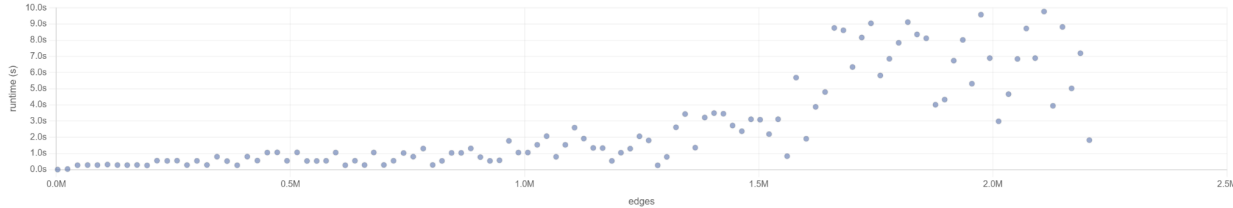
A*: Gap ≈ 0 -13.6% (mean $\approx 5.1\%$) — admissible heuristic, not always optimal on all tie-break configurations

Bellman-Ford: Gap = 0% always (same relaxation, exact optimal)

ACO: Gap ≈ 188 -1,683% (mean $\approx 735\%$) — highly suboptimal, stochastic

Efficiency ratio (path quality per second)
 $\eta = 1 / (Gap\% + 1) / Runtime_s \rightarrow A^*$ dominates at scale

Graph attribute correlation with runtime



Dijkstra runtime vs edges: Pearson $r \approx 0.89$ (strong). Node count and edge count are co-linear ($r \approx 0.9999$), so either attribute predicts runtime equally well. Avg degree and circuitry are nearly constant across scale — they do not predict runtime variation independently.